

GetDraft — what the app is and how it works

A description of the product and the system behind it, for a developer taking on the iOS build. It covers what the app does, how the pieces fit, how to run it, and the behaviours that are specific to this codebase.

PLATFORM	ANDROID	IOS	BACKEND	REVISED
Expo SDK 54 · RN 0.81	shipping · v1.0.0 (29)	never built	live	13 Aug 2026

The iOS app has never been built

Not once — never compiled, never installed, never opened. Nobody knows whether it launches. Everything shipped so far is Android. The shared JavaScript is healthy and verified on a real device, but no iOS-specific configuration has ever been exercised.

1 • The product

GetDraft connects athletes with recruiters and coaches. The core interaction is a swipe deck: you see a profile card and either **Draft** it (interested) or **Pass**. When two people Draft each other it becomes a match and a conversation opens.

The vocabulary is product-specific and appears throughout the code and the UI. It is worth keeping consistent:

TERM	MEANING
Draft	A right swipe. Never “like”.
Pass	A left swipe. Never “reject” or “X”.
Super Draft	A scarce, high-priority Draft. Capped on every plan.
Draft Board	The matches screen.
Draft Score	Ranking metric behind the leaderboard.
“It’s a Draft!”	The match celebration.

Four roles, effectively four different apps

Role is chosen at signup and decides navigation, permissions and which tabs exist. It is not cosmetic, and it is enforced server-side.

- **Athlete** — builds a profile, swipes, gets ranked.
- **Recruiter / Coach** — browses athletes, sends outreach.
- **Parent** — oversight only. Parents cannot swipe; a 403 there is correct behaviour, not a bug.
- **Admin** — provisioned in the database only, never self-assignable.

Minors are a first-class concern

Many athletes are under 18, and two mechanisms exist because of it. Both are load-bearing and enforced on the server.

- **Guardian consent.** An under-18 athlete lands in `pending_guardian` and is blocked from feature endpoints until a parent scans their QR code, answers a questionnaire and records a consent video. Enforced by `ActivationGuard`.
- **Identity verification (KYC).** Handled by Dikit, mandatory during onboarding. The only way past it is "Finish later — log out".

This is also why an adult date of birth matters when testing: a teenage birthday correctly lands in the guardian-approval wait, which looks like a dead end if you are not expecting it.

2 • Architecture

LAYER	STACK	WHERE
Mobile app	Expo SDK 54, React Native 0.81, expo-router 6, Redux	repo root
Backend API	NestJS 11 on Fastify, 18 modules, ~112 endpoints	<code>backend/</code>
Database	Supabase Postgres (auth, storage, realtime)	managed
ORM	Prisma 6 (Discover only) + supabase-js (everything else)	<code>backend/</code>
Hosting	Railway	managed

The app never talks to the database

Every request goes through the NestJS API, which holds the `service_role` key. The app carries only the public `anon` key, and that key has had all table privileges revoked. Direct Supabase queries from the app will fail, and that is deliberate.

The response envelope

Successful responses are wrapped:

```
{ "statusCode": 200, "data": { ... } }
```

Except when the payload contains `cards`, `checkoutUrl` or `statusCode` — those pass through raw. Reading `data.data` on one of those returns `undefined`, which produces a plausible-looking success that renders nothing. The Discover feed is the common case:

```
// services/discover.ts
return (data?.data ?? data) as FeedResponse;
```

Errors are always shaped:

```
{ "statusCode": 400, "message": "...", "error": "Bad Request", "timestamp": "..." }
```

3 • Running it

```
git clone https://github.com/ProgixDev/getdraft.git
cd getdraft
npm install
cd backend && npm install && npx prisma generate && cd ..
```

Node 20+. `prisma generate` is not optional — the backend will not typecheck without it. Then drop in the env files you were sent: `app.env` → `.env` at the repo root, `backend.env` → `backend/.env`.

The app points at the production API by default, so the backend is optional locally. To run it: `cd backend && npm run start:dev` (port 3000) and set `EXPO_PUBLIC_API_URL=http://localhost:3000/api`.

Expo Go is not enough — the app uses native modules (Stripe, WebView, camera, updates), so it needs a development build.

The baseline, so you can tell what you changed

```
npx tsc --noEmit                # app      → 0 errors
npx tsc --noEmit -p backend/tsconfig.json  # backend → 0 errors
cd backend && npx jest           # 90 tests → all pass
npx expo lint                   # 0 errors, warnings only
```

Test login

A sandboxed number — no SMS is sent and no credit is consumed:

```
Phone: +213558780131
Code: 123456
```

Real phone numbers currently cannot receive a code: the SMS account has no balance. Email signup works normally.

4 • External services

SERVICE	USED FOR	STATE
Supabase	Database, auth, storage	Live · free plan
Railway	Backend hosting	Live
Stripe	Subscriptions, Draft packs	LIVE keys — real money
Prelude	Phone OTP	Working · EUR 0.00 balance
Resend	Email + Supabase auth SMTP	Live
Didit	Identity verification	Live
Mapbox	Globe tab, location search	Live
Expo / EAS	Builds, OTA updates	Live

Two things worth knowing before you touch either

Stripe keys are live. Completing a purchase while testing charges a real card. Opening the payment sheet is free; confirming is not.

Supabase is on the free plan — no backups, 1 GB of storage in total, and the project suspends itself after about a week of inactivity. It has already done that once: the project paused, Supabase withdrew its DNS, and the whole app went down until it was moved.

There is no staging. The database, Stripe account and SMS provider you develop against are the live ones. Create throwaway accounts through the API rather than editing rows.

5 • Behaviours specific to this codebase

These are not general React Native advice — they are decisions and traps particular to this project, each of which has already cost time here.

User media is served through signed URLs

The `avatars`, `photos`, `videos` and `posts` buckets are private. Every API response rewrites stored media into a signed URL through a global interceptor, so no endpoint has to remember to do it. `sports` stays public — product icons, not user media.

Never persist a URL containing `/object/sign/`. The client receives a signed URL and can hand it straight back on the next save; storing it would write a token that expires, and the photo would silently disappear a week later. The interceptor strips tokens from inbound bodies, so this is handled — but do not add a write path that bypasses it.

`MEDIA_SIGNING=off` disables the whole mechanism if it ever misbehaves.

expo-image, never react-native's Image

React Native's `Image` decodes remote photos at full resolution. A 320 kB upload at 2048×1536 is 12 MB of bitmap; a grid of them overruns the Android heap and the process is killed. That presents as a grey screen and `ErrorBoundary` cannot catch it, because the crash is native. Twelve screens were converted for this reason.

Always capture the useFonts flag

`const [fontsLoaded] = useFonts(...)`, and hold rendering until it is true. Discarding it means text is measured in the fallback face and painted in the real one — a button reading “Skip” rendered as “**Ski**” for exactly this reason. Every `FontFamily` a screen uses must also be in that screen's own `useFonts` call.

EXPO_PUBLIC_* must reach the cloud build

`.env` is gitignored, and EAS uses `.gitignore` to decide what to upload, so `.env` never reaches a cloud build. The Mapbox token lived only there, which meant the Globe tab and all location search were dead in *every build ever shipped* — silently, because the Globe showed a spinner rather than an error.

It now lives as an EAS environment variable in the `preview` and `production` environments (not in `eas.json` — GitHub's secret scanner rejects Mapbox tokens in the repo). Worth confirming with `eas env:list --environment production` before a release build.

OTA updates, and the fingerprint that strands them

`expo-updates` is configured with a fingerprint runtime version and per-profile channels, so JavaScript-only fixes ship in about a minute without a rebuild. Editing `eas.json` changes that fingerprint, and an update then silently never reaches builds made before the edit. The About screen prints `build <id>` or `build embedded` so you can tell which bundle a device is actually running.

6 • Current iOS state

What has already been prepared, so you are not redoing it:

ITEM	STATE
Bundle identifier	Set — <code>com.getdraft.app</code> , matches Android
Permission strings	Written — <code>ios.infoPlist</code> in app.json
Privacy manifest	Declared — <code>ios.privacyManifests</code> , generated at prebuild
Encryption declaration	Set — <code>ITSApUsesNonExemptEncryption: false</code>
In-app purchases	Gated off for iOS — see below
Apple team	<code>89884BGNZR</code> in <code>eas.json</code> — confirm it is yours to use
APNs push credentials	Not configured — the only push gap; the server side is done

Why purchases are switched off on iOS

The app sells subscriptions and Draft packs through Stripe's native PaymentSheet — a complete third-party purchase flow for digital goods inside the app, which is what Apple requires StoreKit for. `PURCHASES_ENABLED` in `constants/purchases.ts` is now `Platform.OS !== 'ios'` , so every purchase entry point disappears on iOS: the per-plan Upgrade buttons, "Buy more Drafts", and the `/buy-swipes` route, which redirects rather than rendering so a deep link cannot reach the sheet either.

Plans stay visible and cancel/resume keep working — managing an existing subscription is not selling one. Android is unchanged and still sells through Stripe. Reverting is one line if StoreKit gets built.

7 • Access you will need

The env files cover the services themselves. These are accounts someone has to grant:

ACCOUNT	FOR	OWNER
GitHub · ProgixDev/getdraft	The code	Progix
Apple Developer	Signing, App Store Connect	client
Expo / EAS	Builds and OTA updates	getdraftprogix
Supabase	Database, storage, auth settings	client
Stripe	Products, prices, webhooks	client
Railway	Backend env and deploys	third party
Didit / Prelude / Resend / Mapbox	Dashboards, quotas, billing	client

State of the codebase

The backend is verified end to end against production: all endpoints answering, KYC creating real sessions, the Stripe contract correct with no free-upgrade path, websocket auth rejecting unauthenticated sockets, uploads and MIME restrictions enforced. Nine security holes were closed, each confirmed against the live database. Onboarding completes on a real Android device. Typecheck is clean on both sides and 90 backend tests pass.

The shared JavaScript is in good shape. The work ahead is iOS-specific configuration, not repairing the product.